SIMATS ENGINEERING

CSA02 - C Programming

ASSIGNMENT 4

CO4 | 50 marks

Modelling Real-World Data with Structures and Unions

Data models, memory layouts, implementation and validation

Complete solutions for Tasks 1-6

Student Name: PROBIR SAHA SHOHOM

Register Number: 1911260319

GitHub Repository:
Probir127/CSA02-1911260319-Assignment4

Student and execution details

Student: PROBIR SAHA SHOHOM. Register number: 1911260319. Course: CSA02 - C Programming. Assignment4, CO4, 50marks.

Repository: https://github.com/Probir127/CSA02-1911260319-Assignment4

Modelling real-world data with structures and unions. This report covers all six tasks, the three supplied completed-code exercises, memory layouts, diagnoses, pseudocode, measured outputs and alternative designs. All assignment programs are C17. The gcc command on this Mac invokes Apple Clang21.0.0 (arm64); compiler.txt records the version. Commands use -Wall -Wextra -Wpedantic, and the shell runner also executes sanitizer checks. Fixed synthetic records make the demonstrations reproducible; memory addresses vary by run.

Pseudocode uses named ALGORITHM, INPUT, OUTPUT, BEGIN/END and structured IF/FOR blocks. The separate SIMATS pseudocode style guide is referenced by the brief but was not attached or publicly located, so exact typography-specific conformity to that separate guide has not been verified. Output previews are labelled renderings of saved execution transcripts; personal terminal screenshots and faculty submission remain separate steps.

Task 1 - Data design review (6 marks)

1.1 Diagnose the poor design (2 marks)

Deficiency	What can go wrong	Repair
Related values in parallel arrays	Sorting age alone attaches another person's phone or BP to the wrong name	One Patient record contains all related fields
phone[10] for ten digits	The terminating null byte has no space; strcpy writes beyond the row	phone[11], ten validated digits, keep leading zeroes
One float for BP	Two independent measurements cannot be reconstructed reliably from 120/80 or a decimal encoding	BloodPressure has systolic and diastolic members
visitDate[8] as unvalidated text	Common dates need ten visible characters plus a terminator; malformed dates can be accepted	Date contains day, month and year, with calendar validation
No count capacity check	The 101st patient writes beyond the arrays and may corrupt other records	Check count before the copy
Unbounded strcpy on names	A long input can overwrite the next field or another row	Validate bounded, null-terminated source records before structure assignment
Mutable global arrays/count	Any source file with access can accidentally change the register; independent registers are awkward	Caller owns a PatientRegister and passes its pointer
No duplicate identity or field checks	The same ID may be registered twice; impossible dates or nonsensical data can enter the system	Positive unique ID and explicit field validation

These are data-integrity problems, not simply syntax errors. Parallel arrays can be correct if every operation maintains their shared index, but the junior design provides no encapsulation to enforce that rule.

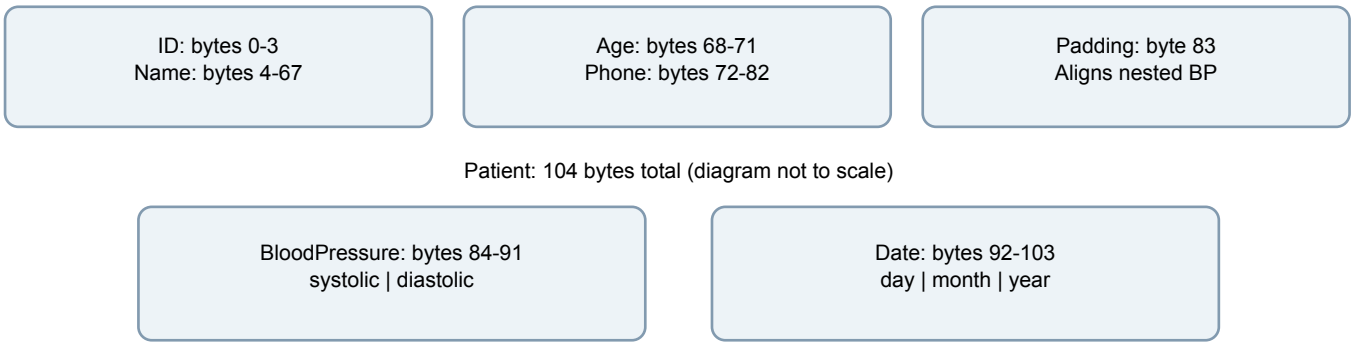
1.2 Repair with nested structures and a pointer interface (3 marks)

Patient contains id, name[64], age, phone[11], BloodPressure bp and Date visited. PatientRegister owns Patient patients[100] and a size_t count. The shared Date type is reused in the courier and canteen tasks. addPatient(PatientRegister *r, const Patient *p) checks pointers, capacity, field validity and duplicate ID before copying *p into the next slot and incrementing count. The source is const because adding a record must not modify it. No write occurs on validation failure.

The demonstration accepts age 0..130, systolic 1..300, diastolic 1..200 with systolic>=diastolic, and calendar dates in 1900..9999. These are explicit classroom input bounds, not a medical interpretation or diagnosis. Name length is at most 63 characters and phone contains exactly ten digits. memchr first establishes a terminator within each bounded array before strlen is used. Actual interactive input would need its own bounded parser; the program constructs and checks synthetic records directly.

```
ALGORITHM AddPatient
INPUT: register pointer R, patient pointer P
OUTPUT: success or rejection
BEGIN
    IF a pointer is invalid OR R.count = capacity THEN RETURN rejection
    IF P has invalid fields THEN RETURN rejection
    FOR each existing patient in R
        IF existing.id = P.id THEN RETURN rejection
    END FOR
    R.patients[R.count] <- complete value of P
    R.count <- R.count + 1
    RETURN success
END
```

Measured Patient size is 104 bytes. The field sizes total 103 bytes, and one padding byte appears before BloodPressure at offset 84:



The two initial records sort from Mina(age42), Arun(age19) into Arun, Mina while their associated fields stay together. The tests reject a duplicate ID, short phone, invalid leap date, invalid BP ordering, missing name terminator and the 101st insertion.

```
2 Arun age=19 phone=9876543210 BP=110/70 date=09/11/2026
1 Mina age=42 phone=0123456789 BP=120/80 date=10/11/2026
sizeof Patient=104; 100 records=10400 bytes; count=100
Patient offsets: id=0 name=4 age=68 phone=72 bp=84 date=92
PASS: associated fields, invalid data, duplicate ID and capacity boundary.
```

1.3 Design defence (under 100 words; 1 mark)

A Patient structure keeps each person's name, age, phone, blood pressure and visit date together. Sorting moves complete records, so their fields cannot become misaligned. Functions accept a pointer to one record or register, avoiding long parameter lists and hidden global state. Nested types reuse date and blood-pressure rules. Adding another field changes the record definition rather than introducing another parallel array that every loop must remember to move. Validation at insertion preserves the register's invariants.

Alternative: structure of arrays

For the same repaired fields, separate member arrays require a logical payload of 103*n bytes on this platform, versus 104*n bytes for Patient[n], excluding any containing object's alignment and the count. The junior arrays use less memory partly because they represent less data and have inadequate buffers, so that is not a fair like-for-like saving. For 100 equivalent records, payload is about 10300 versus 10400 bytes.

A whole-record swap conceptually makes three copies of sizeof(Patient), or 312 bytes here. Swapping every corresponding member of a structure-of-arrays design touches the equivalent member payload, about 309 bytes, plus control operations; moving only age corrupts the relationship. Actual machine traffic can differ after optimization. For b bytes per complete record and O(n log n) comparison sorting, movement can cost O(b*n log n); insertion sort used here is O(n^2) comparisons/moves worst-case, O(n) on sorted data, with one record temporary. Parallel arrays are useful for scans of a single field, vectorized numerical processing and cache efficiency. Sorting an index array can avoid moving either representation's large records, at O(n) extra index storage.

Task 2 - Courier nested structures (8 marks)

2.1 Entities and nesting (2 marks)

Entity	Fields / relationship
Date	day, month, year
Address	city[32], pin[7]
Person	name[64], nested Address
Parcel	ID, weight in grams, booking Date, sender Person, receiver Person, service enum, charge in integer paise
Daily bookings	Parcel bookings[100] plus count

PIN is textual in the full design because it is an identifier, not a quantity; char[7] holds six digits plus null. Exact whole grams avoid float comparisons for the full application's weight. NORMAL and EXPRESS represent the finite service choices. Charge is stored in long long paise; the brief supplies no tariff, so the example stores illustrative charges rather than inventing a pricing formula. The provided complete-code exercise retains its int PIN and float weight exactly as supplied.

2.2 Structure definitions (2 marks)

```
typedef struct { char city[32], pin[7]; } Address;
typedef struct { char name[64]; Address address; } Person;
typedef enum { NORMAL, EXPRESS } Service;
typedef struct {
    unsigned id, weight_grams;
    Date booked;
    Person sender, receiver;
    Service service;
    long long charge_paise;
} Parcel;
Parcel bookings[100];
```

The booking list is caller-owned. A counter application would validate unique positive ID, nonempty bounded strings, six PIN digits, valid date/service, positive weight and non-negative charge before insertion. This task's program uses fixed valid fixtures and calendar assertions; it is not an arbitrary-input parser.

2.3 Operation pseudocode (3 marks)

```
ALGORITHM SortByWeight
INPUT: parcel array P with N records
OUTPUT: P in ascending weight order
BEGIN
    FOR I <- 0 TO N - 2
        LIGHTEST <- I
        FOR J <- I + 1 TO N - 1
            IF P[J].weight < P[LIGHTEST].weight THEN
                LIGHTEST <- J
            END IF
        END FOR
        IF LIGHTEST != I THEN
            TEMP <- P[I]
            P[I] <- P[LIGHTEST]
            P[LIGHTEST] <- TEMP
        END IF
    END FOR
END

ALGORITHM ListByReceiverCity
INPUT: parcel array P, count N, target CITY
OUTPUT: all parcels addressed to CITY
BEGIN
    FOR I <- 0 TO N - 1
        IF CompareStrings(P[I].receiver.address.city, CITY) = 0 THEN
            OUTPUT P[I]
        END IF
    END FOR
END
```

Selection sort performs n(n-1)/2 weight comparisons and at most n-1 swaps: O(n^2) time, O(1) record temporaries. It is not generally stable for equal weights; if booking order among ties matters, use stable insertion sort or add a booking sequence tie-breaker. The city scan costs O(n*L) in the worst case for city length L, with constant auxiliary storage; it uses exact case-sensitive matching here.

2.4 Access paths and execution (1 mark)

For the full data model, the third parcel's receiver PIN is bookings[2].receiver.address.pin. Given Parcel *p=bookings, it is (p+2)->receiver.address.pin, equivalently p[2].receiver.address.pin. In the supplied smaller model it is list[2].to.pincode or (p+2)->to.pincode with p pointing to struct parcel.

The supplied exercise fills only its TODOs and prints:

```
502  1.2 kg  Chennai-600077  12/10/2026
503  2.8 kg  Coimbatore-641001  13/10/2026
501  4.5 kg  Madurai-625001  12/10/2026
```

The two swaps each use a temporary initialization and two assignments. With sizeof(struct parcel)=44, the conceptual payload is 3*44=132 bytes per swap and 264 bytes for both; this is a source-level copy count, not a hardware memory-traffic measurement. Three comparisons select the three records.

```
Rich parcel size=240; flat size=240; comparisons=3 swaps=2
Supplied parcel size=44; logical bytes per swap=132; total two swaps=264
Rich parcel logical bytes per swap=720
Third receiver PIN via dot=625001; via pointer=625001
Receiver city filter: Chennai
ID=502 weight=1200 grams
PASS: whole-record sort and receiver-city filter.
```

Alternative: flattened fields

FlatParcel replaces booked.day/month/year with day/month/year and replaces the Person/Address nesting with sender_name, sender_city, sender_pin and the corresponding receiver fields. A path such as p->receiver.address.pin becomes p->receiver_pin. Both measured layouts are 240 bytes with this field order and ABI: nesting itself does not require an extra pointer or dynamic allocation. Padding and member order can make other designs differ, so sizeof is measured rather than assumed. Access is O(1) in either layout. Reusing Person and Address gives one definition and one validation interface, which is the main design advantage of nesting.

Task 3 - Unions and memory layout (9 marks)

3.1 Compare all-fields and tagged-union packets (3 marks)

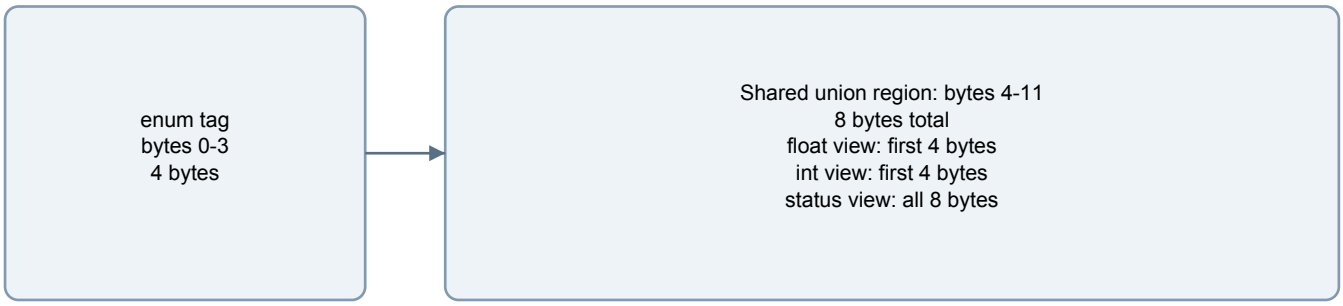
An all-fields structure contains float celsius, int percent and char status[8] simultaneously. A tagged packet contains an enum kind and a union of those same alternatives. Since each packet carries exactly one value, the union shares storage efficiently; the tag records which interpretation the program may use. Updating a packet must update the tag and the corresponding payload as one logical operation.

An all-fields design can be useful if all three measurements are needed together. Here two fields would be irrelevant in every record. For a fair semantic comparison, TaggedAll adds an enum tag to the all-fields design, so both representations identify the active value explicitly.

3.2 Predict and measure sizes (2 marks)

The prediction assumes int, float and enum each occupy four bytes with four-byte alignment. The union needs enough space for its largest member, the eight-byte status buffer, rounded to the union alignment. The tag starts at offset0; the union starts at offset4. Total12 is already a multiple of four, so this layout needs no trailing padding.

Design	Predicted bytes	Measured bytes
AllFields (float + int + char[8])	16	16
TaggedAll	20	20
Value union	8	8
Tagged Packet	12	12



The tag selects ONE interpretation of the shared region.

Tagged packet size: 4 + 8 = 12 bytes on this platform.

The structure is 4+max(4,4,8)=12, not 4+4+4+8. '64-bit machine' does not imply every member is eight bytes; pointer size is a separate property. Other ABIs, enum options, alignments or member types may change these results. For 10000 records the tagged union uses120000 bytes versus160000 for untagged AllFields or200000 for TaggedAll: savings40000 or80000 bytes respectively.

3.3 Tag-directed printing and validation (3 marks)

The completed show function switches on the tag and reads only its matching member, printing temperature to one decimal place, percent with a literal percent sign, or a status string. A default branch handles an invalid tag. The separate constructor set_status accepts at most seven characters, initializes the packet and terminates the buffer; an eight-character input is rejected without changing the previous packet. No dynamic allocation is needed for each record.

```
sizeof union = 8, sizeof struct reading = 12
Temp 31.5 C
Humidity 68 %
Status OK
```

3.4 Inactive-member experiment (1 mark)

After storing31.5f, this host prints1107034112 when reading the same union through its int member. The little-endian float object bytes are00 00 fc 41, corresponding to IEEE binary32 bits0x41fc0000. This is a reinterpretation of representation, not the numerical conversion of31.5 to31 or a valid humidity reading. C's union rules allow this form of representation reinterpretation, but the value is not portable and some representations could be traps. It is inaccurate to promise the same integer on every system or to call it a legitimate tag-independent accessor. The normal application always follows the tag.

```
int=4 float=4 enum=4 pointer=8
all-fields=16 tagged-all=20 union=8 tagged-union=12
union alignment=4 packet alignment=4 type offset=0 value offset=4
10000 packets: all-fields=160000 tagged-all=200000 tagged-union=120000 bytes
Misuse experiment on this platform: stored temperature=31.5; humidity member=1107034112
Float object bytes: 00 00 fc 41
Inactive-member result is representation-dependent, not a humidity conversion.
PASS: status length7 accepted; length8 rejected without changing packet.
```

Alternative storage

Three separate typed arrays can use approximately4*T+4*H+8*S payload bytes for T temperature, H humidity and S status values on this ABI. Reading a known typed index is O(1). To reconstruct one chronological mixed stream, add a type/index mapping or timestamps; that metadata may erase the raw payload saving. Scanning one measurement type can have better locality.

A heap allocation per packet plus a void pointer still needs an external tag, a defined ownership rule and correct casts; void* does not remember a payload type. On this host the pointer alone uses8bytes, plus a tag, payload, alignment and allocator metadata. Access is O(1) after locating the packet but adds indirection and usually poorer locality. Allocator cost is implementation-dependent, with fragmentation and allocation failures to handle. A contiguous tagged-union array has predictable12-byte stride here, O(1) indexed access and O(n) total storage, making it appropriate for this constrained fixed-size packet stream. Its raw memory image is not a portable network serialization format; a wire protocol must define byte order, tags and encoding explicitly.

Task 4 - Structure forensics (7 marks)

4.1 Defects (2 marks)

addInterest(struct account a, float rate) receives a value copy. Assigning a.balance modifies that copy and does not write back to list[i]. Separately, p is a pointer; p.number and p.balance are invalid member access expressions. Use p->number and p->balance, or (*p).number and (*p).balance. The compile-only reproduction includes stdio.h and records both member-access errors:

```
Task04/compile_error.c:10:24: error: member reference type 'struct account *' is a pointer; did you
     mean to use '->'?
   10 |     printf("%d %f\n", p.number, p.balance);
      |                       ~^
      |                        ->
Task04/compile_error.c:10:34: error: member reference type 'struct account *' is a pointer; did you
     mean to use '->'?
   10 |     printf("%d %f\n", p.number, p.balance);
      |                                 ~^
      |                                  ->
2 errors generated.
```

4.2 Construct address evidence (2 marks)

The runnable diagnosis fixes the printing syntax but retains the ineffective by-value operation. It prints the original record's address and the callee's &a with %p and correctly cast pointer arguments. The addresses differ while both objects are alive. The local-copy address can be reused on later calls, which does not imply that the same object persists between calls. Exact addresses vary from run to run; their inequality and the unchanged balances are the relevant evidence.

```
number=101 original=0x16af969a0 local-copy=0x16af96998 different=1
number=102 original=0x16af969a8 local-copy=0x16af96998 different=1
number=103 original=0x16af969b0 local-copy=0x16af96998 different=1
After by-value: 5000.00 12000.00 800.00
Pointer repair: 101 5200.00; returned copy: 5200.00
Pointer repair: 102 12480.00; returned copy: 12480.00
Pointer repair: 103 832.00; returned copy: 832.00
sizeof account=8 pointer=8; conceptual two-copy payload=16
PASS: original unchanged by value; pointer and return-copy repairs agree.
```

4.3 Repair using a structure pointer (2 marks)

```
void addInterest(struct account *a, float rate) {
    a->balance += a->balance * rate / 100;
}
/* For each valid element: */
addInterest(&list[i], 4);
```

The pointer value is still passed by value, but it designates the original array element. Dereferencing it writes that object. The three results5200,12480,832 are asserted and match a return-copy alternative. These functions have the precondition of a valid account pointer and the fixed test rate. Float is retained to diagnose the supplied exercise; an actual monetary system should define rounding, range checks and an exact monetary representation instead of inheriting this classroom float model.

4.4 Passing guideline and alternative (1 mark)

Use by-value parameters for small independent snapshots, by-pointer parameters to update the caller's object, and const pointers to read a potentially large object without copying it. Pointer interfaces require documented validity/lifetime and ownership; const prevents modification through that pointer, not through every possible alias.

The return-copy alternative is list[i]=returned(list[i],4), where returned modifies its local account and returns it. At the source-level model this passes and returns sizeof(account) bytes,16 conceptual bytes for an8-byte record, versus passing an8-byte pointer. For a variable record size b, the copy work/state is O(b), versus O(1) for the pointer value, while the same field arithmetic occurs in both. With a fixed type both are O(1). ABIs may pass/return small records in registers and optimizers may eliminate copies, so neither exact stack usage nor a speed win follows from these source counts. Use mutation semantics and interface clarity first; measure generated code if performance matters.

Task 5 - Storage classes and counters (7 marks)

5.1 Four implementations and outputs (3 marks)

An automatic local initialized to1000 starts again on every call, so it cannot provide persistent token numbers. A static local initialized to1000 retains its updated value throughout program execution. The extern version defines shared_counter once in counters.c and declares it in counters.h; demo.c and other.c call the same token function, demonstrating sharing across source files. The register keyword is shown on both a local counter and a printing-loop variable: it does not create persistence, so a register local initialized on every call also repeats1001.

The supplied completed-code exercise prints:

```
static: 1001 1002 1003
auto  : 1001 1001 1001
```

The complete experiment prints:

```
automatic: 1001 1001 1001
static: 1001 1002 1003
```

```
extern across two callers: 1001 1002 1003
register-local: 1001 1001 1001
static pair: 1001 1001 1002 1002
extern pair: 1001 1001 1002 1002
automatic pair still resets: 1001 1001 1001 1001
caller-owned pair with register loop: 1001 1001 1002 1002
single printf argument-order experiment: 1001 1002 1003
PASS: independent caller state and overflow rejection.
```

The baseline static/global examples use only a few calls within int range. The recommended caller-owned implementation also checks INT_MAX and rejects overflow without changing counter or output. Counters restart when the process restarts; static storage does not mean disk persistence.

5.2 Two independent counters and change effort (2 marks)

Define kind0=veg and kind1=non-veg. Alternating0,1,0,1 should yield1001,1001,1002,1002. The static pair and extern pair achieve that using two state elements. The automatic pair demonstration still returns1001 every time; adding a second automatic variable does not repair its lifetime. Moving both counters into the caller and passing the selected address provides the required lifetime and isolation.

Version	Concrete state/API changes made	Result
Automatic local	Replace per-call local state with one caller-owned int[2]; change function to accept a counter pointer and output pointer; change increment to dereference it	Independent1001,1001,1002,1002; state survives for caller's lifetime
Static local	Replace one static int by a static int[2]; add a kind parameter and bounds check; index the chosen element	Same independent sequence
extern global	Add one int[2] definition in counters.c and matching extern declaration in the header; add kind parameter/bounds check to pair function; callers supply kind	Same independent sequence across modules
register printing loop	Retain register on the loop index; add one caller-owned int[2] and select counters[i%2] for each call	Same independent sequence; register does not own persistent token state

This records the changed declarations, interfaces and call sites rather than treating a formatting-dependent line count as effort. Baseline and modified functions are retained together so the change is directly inspectable. Separate duplicate functions such as veg_token and nonveg_token could each hide a static, but would duplicate logic and make future counters harder to add. The caller-owned design generalizes without new globals and is easier to reset and test.

5.3 Scope, lifetime, initialization and modification risk (2 marks)

Storage form	Scope / linkage	Lifetime	Initial value without explicit initializer	Main risk
auto local	Block scope, no linkage	Current block invocation	Indeterminate	Reset each call; pointer becomes invalid after lifetime ends
static local	Block scope, no linkage	Entire program	Zero	Hidden shared state, testing/reset and concurrency concerns
extern shared definition	Scope of visible declaration; shared external linkage here	Entire program	Definition with static duration defaults to zero	Any permitted module can modify it; multiple definitions cause linkage problems
register local	Block scope, no linkage	Automatic	Indeterminate	No persistence and no permission to take its address

The examples explicitly initialize token state to1000. An extern declaration normally introduces a reference to the single definition rather than allocating another counter. Uninitialized automatic objects must not be read. register is an implementation-dependent optimization hint; the compiler may ignore the hint, but C17 still prohibits computing the address of a register object. The compile-only negative test demonstrates this rule:

```
Task05/register_error.c:2:40: error: address of register variable requested
    2 | int main(void){register int i=0;int *p=&i;return *p;}
      |                                        ^~
1 error generated.
```

Argument order and pointer-state alternative

printf("%d %d %d", next_token(), next_token(), next_token()) does not specify which argument call is evaluated first. This compiler printed1001,1002,1003 in this run, but another permitted evaluation order may print another permutation. The function executions do not interleave, so this is not the same unsequenced-modification bug as count=count++ +1. Assigning each token in a separate statement before printf makes the association with each printed position explicit and portable.

A caller-owned counter has O(1) update time and one int of storage per independent sequence; its lifetime is whichever owner the program chooses (automatic, static or allocated). Static/global forms also have O(1) operations, but hide ownership and share state. No example is thread-safe merely because its storage duration is static; simultaneous counters need synchronization, and cross-process durable tokens need persistent storage.

Task 6 - Canteen inventory and billing (13 marks)

Stage 1: requirements and invariants (2 marks)

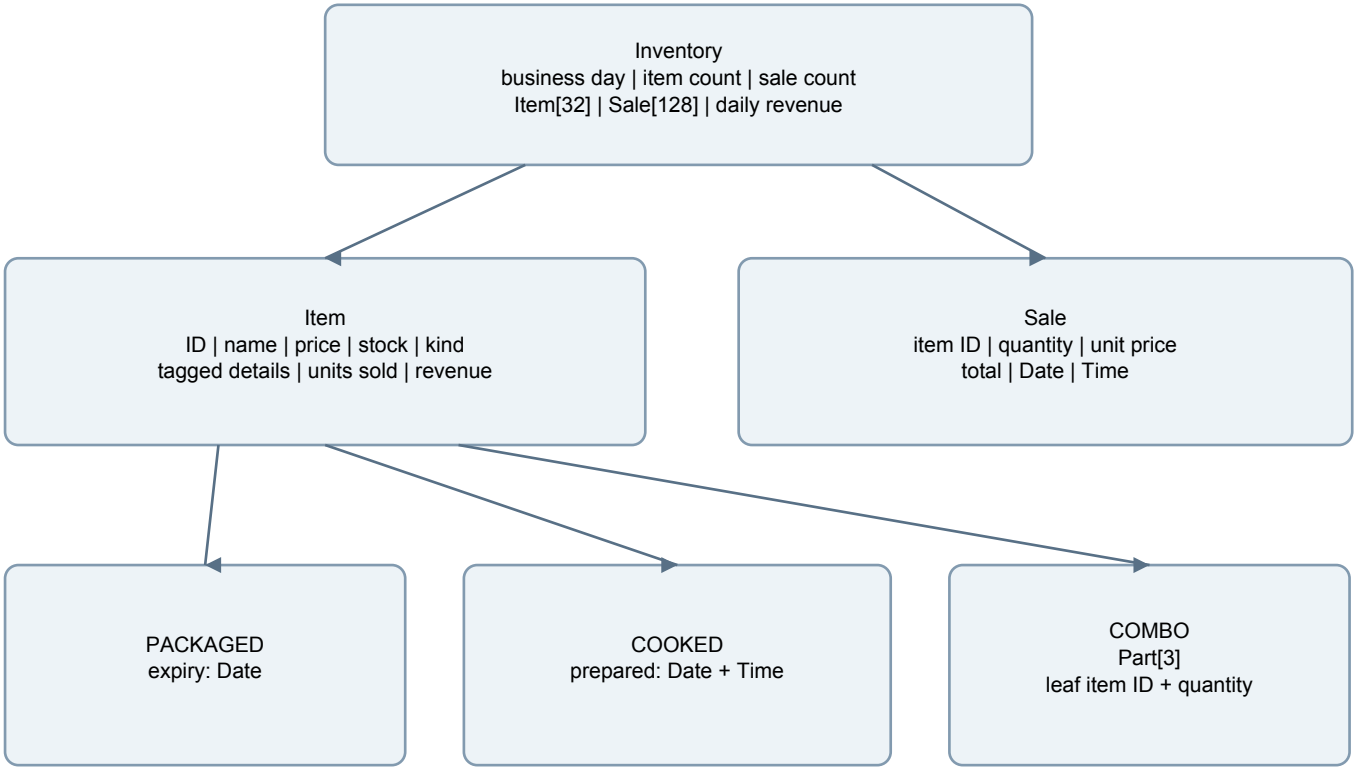
The daily inventory holds at most32 items and128 sale records. Each item has a stable positive ID, bounded name, positive price in paise, kind and kind-specific details. A sale identifies an item, positive quantity, business date and time, and records its unit-price snapshot and total. Packaged items have expiry dates; cooked items have preparation date/time; combos list one to three ingredient entries, each an existing non-combo item ID and quantity.

Requirement	Testable statement
Correct billing	Accepted sale amount equals quantity*price; daily total equals sum of accepted sales
Safe stock	No successful sale makes any stock negative; a failed sale changes no inventory field
Combo accounting	Aggregate repeated ingredient IDs before checking stock; charge combo revenue once, not again to ingredients
Expiry	Warn for packaged dates0..2days ahead inclusive; separately label expired stock and reject its sale
Cooked availability	Demonstration permits cooked items only on their preparation date and at/after preparation time
Daily boundaries	Sale date must equal this inventory's business date
Summary	Descending item revenue, with ascending item ID for ties
Capacity/overflow	Reject full item/sale arrays and arithmetic overflow before mutation

The cooked-item rule is an explicit classroom assumption because the brief supplies no shelf-life policy. It is not a real food-safety guarantee. The program uses a fixed test date10/11/2026 for reproducible outputs. Prices are stored, not inferred from a tariff. Invalid requests return an explanation, and no rejected sale is added to the successful-sale log.

Stage 2: data model (2 marks)

The complete types and APIs are in canteen.h. Item uses a common part (id, name, price, stock, kind, sales totals) and one union: packaged.expiry, cooked.prepared_date/prepared_time, or combo.count/parts[3]. The tag selects the valid alternative. Date and Time are nested structures; Part pairs ingredient ID and quantity; Sale captures the financial/time facts of one transaction. Inventory owns arrays of Item and Sale and their counts.



The measured Item size is112 bytes and Inventory size9768 bytes on this system. The Item layout is id0, name4, price48, stock56, tag60, union64, sold_units96, revenue104. Four padding bytes after name align the eight-byte price; the32-byte union accommodates the largest combo member. Reordering records or sorting a separate index list cannot invalidate ID-based recipes.

Stage 3: critical decisions (2 marks)

A combo has no separately decremented stock balance: availability derives from all its ingredients. Nested combos are rejected, eliminating recursive cycles and simplifying bounded demand calculation. Repeated IDs within a recipe are allowed and combined. For example, a recipe with water quantities1and2 requires3waters per combo, not whichever entry happened to be checked last.

record_sale first builds a scratch demand array, checks every ingredient's availability and quantity, verifies sale-log capacity and all monetary/counter bounds, and only then applies deductions and totals. Nothing in its commit phase can fail. Snapshot assertions verify byte-for-byte equality after rejected operations. This gives atomic behaviour within the single-threaded function; it does not promise thread or crash safety. Multiple terminals would require a lock/transaction around the whole validation-and-commit sequence.

Date validation handles month lengths and Gregorian leap-year rules for1900..9999. Expiry distance is an ordinal-date difference, so the two-day window works across month/year boundaries without timezone or daylight-saving arithmetic. A packaged item expiring today is allowed under the explicit inclusive policy. A previous-day cooked item or future preparation time is unavailable in this demonstration.

No mutable global or static inventory is needed: main owns it and passes pointers. Module-local helper functions use static for internal linkage. Immutable capacity macros constrain the arrays; business state lives in the Inventory. Item insertion establishes valid tags, bounded terminated names, prices <=10^9paise, stock <=10^6 and component quantities <=10^6. Sale quantity <=10^6 makes component products <=10^12 before accumulation into uint64_t;

additions, money products and daily totals are checked. The functions assume subsequent state changes occur through these validated APIs, apart from deliberate boundary-state setup in tests.

Stage 4: function interfaces and pseudocode (4 marks)

```
bool add_item(Inventory *inventory, const Item *item);
SaleResult record_sale(Inventory *inventory, unsigned id,
                       unsigned quantity, Date day, Time time);
void expiry_warnings(const Inventory *inventory);
void print_summary(const Inventory *inventory);
```

add_item copies a valid new record; record_sale mutates stock and sales; reporting accepts const pointers. Clients receive status values instead of partially updated records.

```
ALGORITHM RecordSale
INPUT: inventory I, item ID, quantity Q, date D, time T
OUTPUT: accepted or a specific rejection reason
BEGIN
    IF Q, D or T is invalid OR D is not I.business_day THEN RETURN invalid
    FIND item ID; IF absent THEN RETURN unknown_item
    IF I.sale_count = capacity THEN RETURN log_full
    INITIALIZE all required quantities to zero
    IF item is a combo THEN
        FOR each recipe entry
            FIND its existing leaf ingredient
            ADD Q * entry.quantity to that ingredient's requirement
            IF arithmetic overflows THEN RETURN overflow
        END FOR
    ELSE
        SET sold item's requirement to Q
    END IF
    FOR each required ingredient
        IF it is unavailable on D at T THEN RETURN unavailable
        IF required quantity exceeds stock THEN RETURN insufficient_stock
    END FOR
    COMPUTE charge and CHECK all revenue and unit-count additions
    IF any arithmetic check fails THEN RETURN overflow
    FOR each item
        SUBTRACT its required quantity from stock
    END FOR
    UPDATE sold item's units and revenue and daily revenue
    APPEND one Sale containing ID, Q, unit price, charge, D and T
    RETURN accepted
END

ALGORITHM ExpiryWarnings
INPUT: valid inventory I
OUTPUT: expired and soon-expiring packaged item warnings
BEGIN
    FOR each packaged item P
        DAYS <- Ordinal(P.expiry) - Ordinal(I.business_day)
        IF DAYS < 0 THEN OUTPUT expired warning
        ELSE IF DAYS <= 2 THEN OUTPUT soon-expiring warning
        END IF
    END FOR
END
```

Revenue summary sorts indices rather than moving Item objects. For n items, linear ID lookup is O(n); with at most three recipe entries, demand building and checking remain O(n). The scratch array uses O(n) conceptual storage (fixed32 slots here). A sale-log append is O(1). Insertion sorting the summary indices is O(n^2) worst-case with O(n) index storage; n<=32 makes this simple choice adequate. Expiry scanning is O(n). Larger catalogues could use an ID index and an O(n log n) summary sort.

Stage 5: executed demonstration and tests (3 marks)

Accepted sale	Charge (paise)	Stock effects
Water x2	4000	Water10->8
Breakfast x2	11000	Water8->6; cooked8->4
Cooked item x1	4000	Cooked4->3
Milk x1	2500	Milk4->3
Total	21500	Four successful log entries

A subsequent request for two more breakfast combos needs four cooked units but only three remain. It is rejected before any water or cooked stock is deducted. The expired snack, unknown ID and zero quantity are also rejected. Summary ties at4000paise use item ID to keep output deterministic.

```
Expiry warnings (inclusive today through two days ahead):
EXPIRING: 1 Water (in 2 days)
EXPIRED: 2 Snack (1 days overdue)
EXPIRING: 5 Milk (in 1 days)
Sale item=1 quantity=2: accepted
Sale item=4 quantity=2: accepted
Sale item=3 quantity=1: accepted
Sale item=5 quantity=1: accepted
Sale item=4 quantity=2: insufficient stock
Sale item=2 quantity=1: expired or not available at sale time
Sale item=99 quantity=1: unknown item
Sale item=1 quantity=0: invalid request
Daily summary: descending revenue; equal revenues ordered by item ID
ID=4 Breakfast    sold=2 revenue=Rs 110.00 stock=derived from components
ID=1 Water        sold=2 revenue=Rs 40.00 stock=6
ID=3 Idli         sold=1 revenue=Rs 40.00 stock=3
ID=5 Milk         sold=1 revenue=Rs 25.00 stock=3
ID=2 Snack        sold=0 revenue=Rs 0.00 stock=5
Sales=4 total=Rs 215.00
sizeof Date=12 Time=8 Item=112 Sale=48 Inventory=9768
Item offsets: id=0 name=4 price=48 stock=56 kind=60 union=64 sold=96 revenue=104
PASS: expected totals and stocks; every rejected sale left all inventory bytes unchanged.
```

Additional tests aggregate a repeated ingredient recipe (three waters per combo), reject four combos when only ten waters exist, then accept three combos and leave one water. They verify that only the combo receives revenue. Tests also reject nested or missing recipe references, zero recipe quantities, excessive part count, invalid tag, duplicate item ID, item capacity overflow, sold-item/daily-revenue overflow and a full sale log. Preparation-time, previous-day cooked, expiry-today, expired-ingredient, leap-day and year-boundary cases are covered. Rejected sales compare the complete inventory snapshot before and after.

```
PASS: duplicate component demand aggregated; successful combo revenue counted once.
PASS: rejected sale snapshots unchanged for stock, expiry, time, capacity and overflow.
PASS: invalid recipes/tags/IDs and item capacity rejected; leap/year date boundaries verified.
PASS: expiry today allowed; expired ingredient and previous-day cooked item rejected.
```

Alternative: linked-list inventory and remaining limits

A linked list can prepend an item in O(1), or append in O(1) with a maintained tail, once validation/search costs are excluded. Finding by ID is still O(n). Removing by ID costs O(n) to find the node/predecessor, then O(1) to unlink; removal is only O(1) when the necessary node links are already known. An array appends in O(1) while spare capacity exists and removes an interior record by O(n) shifts. Array capacity growth may require allocation/copying. Both need an O(n) uniqueness search without a separate index.

On this machine one next pointer adds8bytes per node, potentially more after alignment and allocator metadata. Separate nodes can have weaker locality and require allocation-failure/ownership handling. Stable IDs are still necessary for recipes and historical sales; removing a referenced ingredient must be prohibited or handled by an explicit archival rule. The current fixed array is suitable for a small bounded single-terminal exercise.

Remaining limits include in-memory storage lost on exit, no persistence/recovery, no concurrency control, one expiry/preparation batch per item ID, no refunds/taxes and a single business-day summary. Supporting several stock batches would need a Batch entity, per-batch expiry/quantity and an explicit allocation policy. These are future requirements rather than silently assumed capabilities of the implemented model.

References

The supplied CSA02 CO4 Assignment4 document defines the task requirements. WG14 N1570, the public C11 committee draft, was consulted for the relevant C rules retained in these C17 examples, particularly storage duration/linkage, structures/unions, address-of constraints and function-call sequencing. Its section6.5.2.3 footnote95 discusses inactive union-member representation: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf. Local sizeof, offsets, diagnostics and outputs in this report were measured with the compiler recorded in compiler.txt.

Appendix - C source listings

Each listing is labelled with its folder and filename. Related files are included in the companion ZIP and repository. Deliberate error examples are identified in their task explanations. Long lines are visually wrapped; use the original source files for execution.

Task01/patient.c

```
#include <stdio.h>
#include <stddef.h>
#include <string.h>
#include <stdbool.h>
#include <assert.h>
#include "../common/date.h"
#define MAX_PATIENTS 100
typedef struct { unsigned systolic,diastolic; } BloodPressure;
typedef struct {
    unsigned id;char name[64];unsigned age;char phone[11];
    BloodPressure bp;Date visited;
} Patient;
typedef struct { Patient patients[MAX_PATIENTS];size_t count; } PatientRegister;
static bool valid(const Patient *p){
    if(!p||!p->id||!memchr(p->name,'\0',sizeof p->name)||!p->name[0]||p->age>130||
       !memchr(p->phone,'\0',sizeof p->phone)||strlen(p->phone)!=10||!date_valid(p->visited))return
            false;
    for(size_t i=0;i<10;i++)if(p->phone[i]<'0'||p->phone[i]>'9')return false;
    return p->bp.systolic>0&&p->bp.systolic<=300&&p->bp.diastolic>0&&
           p->bp.diastolic<=200&&p->bp.systolic>=p->bp.diastolic;
}
static bool addPatient(PatientRegister *r,const Patient *p){
    if(!r||r->count>=MAX_PATIENTS||!valid(p))return false;
    for(size_t i=0;i<r->count;i++)if(r->patients[i].id==p->id)return false;
    r->patients[r->count++]=*p;return true;
}
static void sort_age(PatientRegister *r){
    for(size_t i=1;i<r->count;i++){
        Patient key=r->patients[i];size_t j=i;
        while(j>0&&r->patients[j-1].age>key.age){r->patients[j]=r->patients[j-1];--j;}
        r->patients[j]=key;
    }
}
int main(void){
    PatientRegister r={0};
    Patient a={1,"Mina",42,"0123456789",{120,80},{10,11,2026}};
    Patient b={2,"Arun",19,"9876543210",{110,70},{9,11,2026}};
    assert(addPatient(&r,&a)&&addPatient(&r,&b));sort_age(&r);
    for(size_t i=0;i<r.count;i++){
        const Patient *p=&r.patients[i];
        printf("%u %s age=%u phone=%s BP=%u/%u
            date=%02d/%02d/%04d\n",p->id,p->name,p->age,p->phone,
               p->bp.systolic,p->bp.diastolic,p->visited.day,p->visited.month,p->visited.year);
    }
    assert(r.patients[0].id==2&&!strcmp(r.patients[0].phone,b.phone));
    size_t saved=r.count;assert(!addPatient(&r,&a));
    Patient bad=a;bad.id=3;strcpy(bad.phone,"123");assert(!addPatient(&r,&bad));
    bad=a;bad.id=3;bad.visited=(Date){29,2,2025};assert(!addPatient(&r,&bad));
    bad=a;bad.id=3;bad.bp=(BloodPressure){80,120};assert(!addPatient(&r,&bad));
    bad=a;bad.id=3;memset(bad.name,'x',sizeof bad.name);assert(!addPatient(&r,&bad));
    assert(r.count==saved);
    for(unsigned id=3;id<=100;id++){a.id=id;assert(addPatient(&r,&a));}
    a.id=101;assert(!addPatient(&r,&a)&&r.count==100);
    printf("sizeof Patient=%zu; 100 records=%zu bytes; count=%zu\n",sizeof(Patient),sizeof
        r.patients,r.count);
    printf("Patient offsets: id=%zu name=%zu age=%zu phone=%zu bp=%zu date=%zu\n",
           offsetof(Patient,id),offsetof(Patient,name),offsetof(Patient,age),offsetof(Patient,phone
               ),offsetof(Patient,bp),offsetof(Patient,visited));
    puts("PASS: associated fields, invalid data, duplicate ID and capacity boundary.");return 0;
}
```

Task02/complete_code.c

```
#include <stdio.h>
struct date    { int day, month, year; };
struct address { char city[20]; int pincode; };
struct parcel  { int id; float weight; struct address to; struct date booked; };
void print_parcel(const struct parcel *p)
{
    // TODO 1: Print id, weight (1 decimal), city, pincode and booking date (dd/mm/yyyy)
    //   using the -> operator
    printf("%d  %.1f kg  %s-%d  %02d/%02d/%04d\n",p->id,p->weight,p->to.city,p->to.pincode,
           p->booked.day,p->booked.month,p->booked.year);
}
/* Selection sort of the parcels in ascending order of weight. */
void sort_by_weight(struct parcel p[], int n)
{
    // TODO 2: For each position i, find the lightest parcel among i..n-1 and swap the
    //   whole structure into position i
    for(int i=0;i<n-1;i++){
        int lightest=i;
        for(int j=i+1;j<n;j++)if(p[j].weight<p[lightest].weight)lightest=j;
        if(lightest!=i){struct parcel temporary=p[i];p[i]=p[lightest];p[lightest]=temporary;}
    }
}
int main(void)
{
    struct parcel list[3] = {
        {501, 4.5f, {"Madurai", 625001}, {12, 10, 2026}},
        {502, 1.2f, {"Chennai", 600077}, {12, 10, 2026}},
        {503, 2.8f, {"Coimbatore", 641001}, {13, 10, 2026}}
    };
    sort_by_weight(list, 3);
    for (int i = 0; i < 3; i++) print_parcel(&list[i]);
    return 0;
}
```

Task02/parcels.c

```
#include <stdio.h>
#include <stddef.h>
#include <string.h>
#include <assert.h>
#include "../common/date.h"
typedef struct { char city[32],pin[7]; } Address;
typedef struct { char name[64];Address address; } Person;
typedef enum { NORMAL,EXPRESS } Service;
typedef struct { unsigned id,weight_grams;Date booked;Person sender,receiver;Service service;long
    long charge_paise; } Parcel;
typedef struct {
    unsigned id,weight_grams;int day,month,year;
    char sender_name[64],sender_city[32],sender_pin[7];
    char receiver_name[64],receiver_city[32],receiver_pin[7];
    Service service;long long charge_paise;
} FlatParcel;
struct simple_address {char city[20];int pincode;};
struct simple_parcel {int id;float weight;struct simple_address to;Date booked;};
int main(void){
    Parcel bookings[100]={0};size_t count=3;
    const char *cities[]={"Madurai","Chennai","Coimbatore"};
    const char *pins[]={"625001","600077","641001"};
    unsigned weights[]={4500,1200,2800};
    for(size_t i=0;i<count;i++){
        bookings[i].id=501+(unsigned)i;bookings[i].weight_grams=weights[i];
        bookings[i].booked=(Date){12+(i==2),10,2026};
        strcpy(bookings[i].sender.name,"Sender");strcpy(bookings[i].sender.address.city,"Chennai");

            strcpy(bookings[i].sender.address.pin,"600077");strcpy(bookings[i].receiver.name,"Recei
            ver");
        strcpy(bookings[i].receiver.address.city,cities[i]);strcpy(bookings[i].receiver.address.pin
            ,pins[i]);
        bookings[i].service=i==1?EXPRESS:NORMAL;bookings[i].charge_paise=10000+(long long)i*1000;
        assert(date_valid(bookings[i].booked));
    }
    unsigned comparisons=0,swaps=0;
    for(size_t i=0;i+1<count;i++){
        size_t lightest=i;
        for(size_t j=i+1;j<count;j++){++comparisons;if(bookings[j].weight_grams<bookings[lightest].
            weight_grams)lightest=j;}
        if(lightest!=i){Parcel
            t=bookings[i];bookings[i]=bookings[lightest];bookings[lightest]=t;++swaps;}
    }
    assert(comparisons==3&&swaps==2&&bookings[0].id==502&&bookings[2].id==501);
    printf("Rich parcel size=%zu; flat size=%zu; comparisons=%u
        swaps=%u\n",sizeof(Parcel),sizeof(FlatParcel),comparisons,swaps);
    printf("Supplied parcel size=%zu; logical bytes per swap=%zu; total two swaps=%zu\n",
           sizeof(struct simple_parcel),3*sizeof(struct simple_parcel),6*sizeof(struct
               simple_parcel));
    printf("Rich parcel logical bytes per swap=%zu\n",3*sizeof(Parcel));
    Parcel *p=bookings;
    printf("Third receiver PIN via dot=%s; via
        pointer=%s\n",bookings[2].receiver.address.pin,(p+2)->receiver.address.pin);
    puts("Receiver city filter: Chennai");unsigned matches=0;
    for(size_t i=0;i<count;i++)if(!strcmp(bookings[i].receiver.address.city,"Chennai")){
        printf("ID=%u weight=%u grams\n",bookings[i].id,bookings[i].weight_grams);++matches;
    }
    assert(matches==1);puts("PASS: whole-record sort and receiver-city filter.");return 0;
}
```

Task03/complete_code.c

```
#include <stdio.h>
enum kind { TEMPERATURE, HUMIDITY, STATUS };
struct reading {
    enum kind type;                 /* tag: tells which union member is valid */
    union {
        float celsius;
        int   percent;
        char  status[8];
    } value;
};
void show(const struct reading *r)
{
    // TODO 1: Use a switch on r->type to print only the valid union member: "Temp 31.5
    //   C", "Humidity 68 %" or "Status OK"
    switch(r->type){
        case TEMPERATURE:printf("Temp %.1f C\n",r->value.celsius);break;
        case HUMIDITY:printf("Humidity %d %%\n",r->value.percent);break;
        case STATUS:printf("Status %.7s\n",r->value.status);break;
        default:puts("Invalid packet tag");break;
    }
}
int main(void)
{
    struct reading log[3] = {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY,    {.percent = 68}},
        {STATUS,      {.status = "OK"}}
    };
    printf("sizeof union = %zu, sizeof struct reading = %zu\n",
           sizeof log[0].value, sizeof log[0]);
    for (int i = 0; i < 3; i++) show(&log[i]);
    return 0;
}
```

Task03/layout.c

```
#include <stdio.h>
#include <stddef.h>
#include <string.h>
#include <stdbool.h>
#include <assert.h>
enum Kind { TEMP,HUM,STATUS };
union Value { float celsius;int percent;char status[8]; };
struct AllFields {float celsius;int percent;char status[8];};
struct TaggedAll {enum Kind type;struct AllFields value;};
struct Packet {enum Kind type;union Value value;};
static bool set_status(struct Packet *p,const char *s){
    if(!p||!s||strlen(s)>7)return false;
    *p=(struct Packet){.type=STATUS,.value.status={0}};strcpy(p->value.status,s);return true;
}
int main(void){
    printf("int=%zu float=%zu enum=%zu pointer=%zu\n",sizeof(int),sizeof(float),sizeof(enum
        Kind),sizeof(void*));
    printf("all-fields=%zu tagged-all=%zu union=%zu tagged-union=%zu\n",sizeof(struct
        AllFields),sizeof(struct TaggedAll),sizeof(union Value),sizeof(struct Packet));
    printf("union alignment=%zu packet alignment=%zu type offset=%zu value offset=%zu\n",
           _Alignof(union Value),_Alignof(struct Packet),offsetof(struct
               Packet,type),offsetof(struct Packet,value));
    printf("10000 packets: all-fields=%zu tagged-all=%zu tagged-union=%zu bytes\n",
           10000*sizeof(struct AllFields),10000*sizeof(struct TaggedAll),10000*sizeof(struct
               Packet));
    struct Packet p;assert(set_status(&p,"ABCDEFG"));assert(!set_status(&p,"ABCDEFGH"));
    assert(!strcmp(p.value.status,"ABCDEFG"));
    union Value misuse={.celsius=31.5f};
    printf("Misuse experiment on this platform: stored temperature=%.1f; humidity
        member=%d\n",misuse.celsius,misuse.percent);
    const unsigned char *bytes=(const unsigned char*)&misuse.celsius;
    printf("Float object bytes:");for(size_t i=0;i<sizeof(float);i++)printf("
        %02x",(unsigned)bytes[i]);putchar('\n');
    puts("Inactive-member result is representation-dependent, not a humidity conversion.");
    puts("PASS: status length7 accepted; length8 rejected without changing packet.");return 0;
}
```

Task04/accounts.c

```
#include <stdio.h>
#include <assert.h>
struct account {int number;float balance;};
static void ineffective(struct account a,float rate,const struct account *original){
    printf("number=%d original=%p local-copy=%p different=%d\n",a.number,(const
        void*)original,(void*)&a,&a!=original);
    assert(&a!=original);a.balance+=a.balance*rate/100;
}
static void addInterest(struct account *a,float rate){a->balance+=a->balance*rate/100;}
static struct account returned(struct account a,float rate){a.balance+=a.balance*rate/100;return
    a;}
int main(void){
    struct account list[3]={{101,5000},{102,12000},{103,800}};
    const float before[]={5000,12000,800},after[]={5200,12480,832};
    for(int i=0;i<3;i++){ineffective(list[i],4,&list[i]);assert(list[i].balance==before[i]);}
    puts("After by-value: 5000.00 12000.00 800.00");
    for(int i=0;i<3;i++){
        struct account alternate=returned(list[i],4);addInterest(&list[i],4);
        assert(list[i].balance==after[i]&&alternate.balance==list[i].balance);
        printf("Pointer repair: %d %.2f; returned copy:
            %.2f\n",list[i].number,list[i].balance,alternate.balance);
    }
    printf("sizeof account=%zu pointer=%zu; conceptual two-copy payload=%zu\n",sizeof(struct
        account),sizeof(struct account*),2*sizeof(struct account));
    puts("PASS: original unchanged by value; pointer and return-copy repairs agree.");return 0;
}
```

Task04/compile_error.c

```
#include <stdio.h>
struct account { int number; float balance; };
void addInterest(struct account a, float rate) {
    a.balance = a.balance + a.balance * rate / 100;
}
int main(void) {
    struct account list[3] = {{101, 5000}, {102, 12000}, {103, 800}};
    struct account *p = list;
    for (int i = 0; i < 3; i++) addInterest(list[i], 4);
    printf("%d %f\n", p.number, p.balance);
    return 0;
}
```

Task05/complete_code.c

```
#include <stdio.h>
/* Issues the next mess token number, starting from 1001. */
int next_token(void)
{
    // TODO 1: Use a static local variable so that the counter keeps its value between
    //   calls
    static int token=1000;
    return ++token;
}
/* The same logic with an automatic variable, for comparison. */
int auto_token(void)
{
    int token = 1000;
    return ++token;
}
int main(void)
{
    int s1 = next_token(), s2 = next_token(), s3 = next_token();
    int a1 = auto_token(), a2 = auto_token(), a3 = auto_token();
    printf("static: %d %d %d\n", s1, s2, s3);
    printf("auto  : %d %d %d\n", a1, a2, a3);
    return 0;
}
```

Task05/counters.c

```
#include "counters.h"
int shared_counter=1000;
int shared_pair[2]={1000,1000};
int external_token(void){return ++shared_counter;}
int external_pair(unsigned kind){return kind<2?++shared_pair[kind]:-1;}
```

Task05/counters.h

```
#ifndef CSA02_COUNTERS_H
#define CSA02_COUNTERS_H
extern int shared_counter;
extern int shared_pair[2];
int external_token(void);
int other_source_token(void);
int external_pair(unsigned kind);
#endif
```

Task05/demo.c

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>
#include <assert.h>
#include "counters.h"
static int automatic_token(void){auto int token=1000;return ++token;}
static int persistent_token(void){static int token=1000;return ++token;}
static int register_token(void){register int token=1000;return ++token;}
static int static_pair(unsigned kind){static int tokens[2]={1000,1000};return
    kind<2?++tokens[kind]:-1;}
static int automatic_pair(unsigned kind){int tokens[2]={1000,1000};return
    kind<2?++tokens[kind]:-1;}
static int args_token(void){static int token=1000;return ++token;}
static bool caller_token(int *counter,int *out){
    if(!counter||!out||*counter==INT_MAX)return false;
    *out=++*counter;return true;
}
int main(void){
    printf("automatic:");for(int i=0;i<3;i++)printf(" %d",automatic_token());putchar('\n');
    printf("static:");for(int i=0;i<3;i++)printf(" %d",persistent_token());putchar('\n');
    int a=external_token(),b=other_source_token(),c=external_token();
    printf("extern across two callers: %d %d %d\n",a,b,c);assert(a==1001&&b==1002&&c==1003);
    printf("register-local:");for(register int i=0;i<3;i++)printf("
        %d",register_token());putchar('\n');
    printf("static pair:");for(unsigned i=0;i<4;i++)printf(" %d",static_pair(i%2));putchar('\n');
    printf("extern pair:");for(unsigned i=0;i<4;i++)printf(" %d",external_pair(i%2));putchar('\n');
    printf("automatic pair still resets:");for(unsigned i=0;i<4;i++)printf("
        %d",automatic_pair(i%2));putchar('\n');
    int counters[2]={1000,1000};
    printf("caller-owned pair with register loop:");
    for(register unsigned i=0;i<4;i++){int
        value;assert(caller_token(&counters[i%2],&value));printf(" %d",value);}
    putchar('\n');assert(counters[0]==1002&&counters[1]==1002);
    printf("single printf argument-order experiment: %d %d
        %d\n",args_token(),args_token(),args_token());
    int limit=INT_MAX,out=0;assert(!caller_token(&limit,&out)&&limit==INT_MAX&&out==0);
    assert(!caller_token(NULL,&out));puts("PASS: independent caller state and overflow
        rejection.");return 0;
}
```

Task05/other.c

```
#include "counters.h"
int other_source_token(void){return external_token();}
```

Task05/register_error.c

```
/* Compile-only constraint violation: the address of a register object is unavailable. */
int main(void){register int i=0;int *p=&i;return *p;}
```

Task06/canteen.c

```
#include "canteen.h"
#include <stdio.h>
#include <string.h>
#include <limits.h>
#include <stdint.h>
static int find_item(const Inventory *inv,unsigned id){
    for(size_t i=0;i<inv->count;i++)if(inv->items[i].id==id)return (int)i;
    return -1;
}
static bool valid_time(Time t){return t.hour<24&&t.minute<60;}
static unsigned minutes(Time t){return 60*t.hour+t.minute;}
static void money(long long p){printf("Rs %lld.%02lld",p/100,p%100);}
bool add_item(Inventory *inv,const Item *item){
    if(!inv||!item||inv->count>=MAX_ITEMS||!date_valid(inv->business_day)||!item->id||
       find_item(inv,item->id)>=0||!memchr(item->name,'\0',sizeof item->name)||!item->name[0]||
       item->price_paise<=0||item->price_paise>1000000000LL||item->stock>1000000||
       item->sold_units||item->revenue_paise)return false;
    switch(item->kind){
    case PACKAGED:if(!date_valid(item->data.packaged.expiry))return false;break;
    case COOKED:
        if(!date_valid(item->data.cooked.prepared_date)||!valid_time(item-
            >data.cooked.prepared_time))return false;
        break;
    case COMBO:
        if(item->stock||item->data.combo.count==0||item->data.combo.count>MAX_PARTS)return false;
        for(size_t j=0;j<item->data.combo.count;j++){
            Part part=item->data.combo.parts[j];int at=find_item(inv,part.item_id);
            if(at<0||inv->items[at].kind==COMBO||!part.quantity||part.quantity>1000000)return
                false;
        }
        break;
    default:return false;
    }
    inv->items[inv->count++]=*item;return true;
}
static bool available(const Item *item,Date day,Time now){
    if(item->kind==PACKAGED)return date_ordinal(item->data.packaged.expiry)>=date_ordinal(day);
    if(item->kind==COOKED)return date_ordinal(item->data.cooked.prepared_date)==date_ordinal(day)&&
        minutes(item->data.cooked.prepared_time)<=minutes(now);
    return false;
}
SaleResult record_sale(Inventory *inv,unsigned id,unsigned quantity,Date day,Time time){
    if(!inv||!quantity||quantity>1000000||!date_valid(day)||!valid_time(time)||
       !date_valid(inv->business_day)||date_ordinal(day)!=date_ordinal(inv->business_day))return
           BAD_REQUEST;
    int at=find_item(inv,id);if(at<0)return UNKNOWN_ITEM;
    if(inv->sale_count>=MAX_SALES)return SALES_FULL;
    Item *sold=&inv->items[at];uint64_t needed[MAX_ITEMS]={0};
    if(sold->kind==COMBO){
        for(size_t j=0;j<sold->data.combo.count;j++){
            Part part=sold->data.combo.parts[j];int ingredient=find_item(inv,part.item_id);
            if(ingredient<0||inv->items[ingredient].kind==COMBO)return BAD_REQUEST;
            uint64_t amount=(uint64_t)quantity*part.quantity;
            if(needed[ingredient]>UINT64_MAX-amount)return AMOUNT_OVERFLOW;
            needed[ingredient]+=amount;
        }
    } else needed[at]=quantity;
    for(size_t i=0;i<inv->count;i++)if(needed[i]){
        if(!available(&inv->items[i],day,time))return UNAVAILABLE;
        if(needed[i]>inv->items[i].stock)return INSUFFICIENT_STOCK;
    }
    if(sold->price_paise>LLONG_MAX/quantity)return AMOUNT_OVERFLOW;
    long long charge=sold->price_paise*quantity;
    if(sold->revenue_paise>LLONG_MAX-charge||inv->total_revenue>LLONG_MAX-charge||
       sold->sold_units>ULLONG_MAX-quantity)return AMOUNT_OVERFLOW;
    /* Commit phase: every validation succeeded; nothing below can reject. */
    for(size_t i=0;i<inv->count;i++)inv->items[i].stock-=(unsigned)needed[i];
    sold->sold_units+=quantity;sold->revenue_paise+=charge;inv->total_revenue+=charge;
    inv->sales[inv->sale_count++]=(Sale){id,quantity,sold->price_paise,charge,day,time};
    return SALE_OK;
}
void expiry_warnings(const Inventory *inv){
    puts("Expiry warnings (inclusive today through two days ahead):");
    for(size_t i=0;i<inv->count;i++){
        const Item *p=&inv->items[i];if(p->kind!=PACKAGED)continue;
        int delta=date_ordinal(p->data.packaged.expiry)-date_ordinal(inv->business_day);
        if(delta<0)printf("EXPIRED: %u %s (%d days overdue)\n",p->id,p->name,-delta);
        else if(delta<=2)printf("EXPIRING: %u %s (in %d days)\n",p->id,p->name,delta);
```

```
    }
}
void print_summary(const Inventory *inv){
    size_t order[MAX_ITEMS];for(size_t i=0;i<inv->count;i++)order[i]=i;
    for(size_t i=1;i<inv->count;i++){
        size_t key=order[i],j=i;
        while(j>0){
            const Item *a=&inv->items[order[j-1]],*b=&inv->items[key];

                if(a->revenue_paise>b->revenue_paise||(a->revenue_paise==b->revenue_paise&&a->id<b-
                >id))break;
            order[j]=order[j-1];--j;
        }
        order[j]=key;
    }
    puts("Daily summary: descending revenue; equal revenues ordered by item ID");
    for(size_t i=0;i<inv->count;i++){
        const Item *p=&inv->items[order[i]];
        printf("ID=%u %-12s sold=%llu
            revenue=",p->id,p->name,p->sold_units);money(p->revenue_paise);
        if(p->kind==COMBO)puts(" stock=derived from components");else printf("
            stock=%u\n",p->stock);
    }
    printf("Sales=%zu total=",inv->sale_count);money(inv->total_revenue);putchar('\n');
}
const char *sale_message(SaleResult s){
    switch(s){case SALE_OK:return "accepted";case BAD_REQUEST:return "invalid request";
    case UNKNOWN_ITEM:return "unknown item";case INSUFFICIENT_STOCK:return "insufficient stock";
    case UNAVAILABLE:return "expired or not available at sale time";case SALES_FULL:return "sale
        log full";
    default:return "amount overflow";}
}
```

Task06/canteen.h

```
#ifndef CSA02_CANTEEN_H
#define CSA02_CANTEEN_H
#include <stddef.h>
#include <stdbool.h>
#include "../common/date.h"
#define MAX_ITEMS 32
#define MAX_SALES 128
#define MAX_PARTS 3
typedef struct {unsigned hour,minute;} Time;
typedef enum {PACKAGED,COOKED,COMBO} ItemKind;
typedef struct {unsigned item_id,quantity;} Part;
typedef struct {
    unsigned id;char name[40];long long price_paise;unsigned stock;
    ItemKind kind;
    union {
        struct {Date expiry;} packaged;
        struct {Date prepared_date;Time prepared_time;} cooked;
        struct {size_t count;Part parts[MAX_PARTS];} combo;
    } data;
    unsigned long long sold_units;long long revenue_paise;
} Item;
typedef struct {unsigned item_id,quantity;long long unit_price,total;Date day;Time time;} Sale;
typedef struct {
    Date business_day;Item items[MAX_ITEMS];size_t count;
    Sale sales[MAX_SALES];size_t sale_count;long long total_revenue;
} Inventory;
typedef enum
    {SALE_OK,BAD_REQUEST,UNKNOWN_ITEM,INSUFFICIENT_STOCK,UNAVAILABLE,SALES_FULL,AMOUNT_OVERFLOW}
    SaleResult;
bool add_item(Inventory *inventory,const Item *item);
SaleResult record_sale(Inventory *inventory,unsigned id,unsigned quantity,Date day,Time time);
void expiry_warnings(const Inventory *inventory);
void print_summary(const Inventory *inventory);
const char *sale_message(SaleResult result);
#endif
```

Task06/demo_data.c

```
#include "demo_data.h"
#include <assert.h>
void setup_demo(Inventory *inv){
    *inv=(Inventory){.business_day={10,11,2026}};
    Item water={.id=1,.name="Water",.price_paise=2000,.stock=10,.kind=PACKAGED,.data.packaged={{12,
        11,2026}}};
    Item
        snack={.id=2,.name="Snack",.price_paise=3000,.stock=5,.kind=PACKAGED,.data.packaged={{9,11,
        2026}}};
    Item cooked={.id=3,.name="Idli",.price_paise=4000,.stock=8,.kind=COOKED,.data.cooked={{10,11,20
        26},{8,0}}};
    Item
        combo={.id=4,.name="Breakfast",.price_paise=5500,.kind=COMBO,.data.combo={2,{{1,1},{3,2}}}}
        ;
    Item
        milk={.id=5,.name="Milk",.price_paise=2500,.stock=4,.kind=PACKAGED,.data.packaged={{11,11,2
        026}}};
    assert(add_item(inv,&water)&&add_item(inv,&snack)&&add_item(inv,&cooked)&&add_item(inv,&combo)&
        &add_item(inv,&milk));
}
```

Task06/demo_data.h

```
#ifndef CSA02_DEMO_DATA_H
#define CSA02_DEMO_DATA_H
#include "canteen.h"
void setup_demo(Inventory *inventory);
#endif
```

Task06/main.c

```
#include "demo_data.h"
#include <stdio.h>
#include <assert.h>
#include <string.h>
static void sale(Inventory *inv,unsigned id,unsigned quantity,SaleResult expected){
    Inventory saved=*inv;
    SaleResult actual=record_sale(inv,id,quantity,inv->business_day,(Time){9,0});
    printf("Sale item=%u quantity=%u:
        %s\n",id,quantity,sale_message(actual));assert(actual==expected);
    if(actual!=SALE_OK)assert(memcmp(&saved,inv,sizeof saved)==0);
}
int main(void){
    Inventory inventory;setup_demo(&inventory);
    expiry_warnings(&inventory);
    sale(&inventory,1,2,SALE_OK);sale(&inventory,4,2,SALE_OK);
    sale(&inventory,3,1,SALE_OK);sale(&inventory,5,1,SALE_OK);
    sale(&inventory,4,2,INSUFFICIENT_STOCK);sale(&inventory,2,1,UNAVAILABLE);
    sale(&inventory,99,1,UNKNOWN_ITEM);sale(&inventory,1,0,BAD_REQUEST);
    assert(inventory.total_revenue==21500&&inventory.sale_count==4);
    assert(inventory.items[0].stock==6&&inventory.items[2].stock==3&&inventory.items[4].stock==3);
    print_summary(&inventory);
    printf("sizeof Date=%zu Time=%zu Item=%zu Sale=%zu
        Inventory=%zu\n",sizeof(Date),sizeof(Time),sizeof(Item),sizeof(Sale),sizeof(Inventory));
    printf("Item offsets: id=%zu name=%zu price=%zu stock=%zu kind=%zu union=%zu sold=%zu
        revenue=%zu\n",
           offsetof(Item,id),offsetof(Item,name),offsetof(Item,price_paise),offsetof(Item,stock),of
               fsetof(Item,kind),offsetof(Item,data),offsetof(Item,sold_units),offsetof(Item,revenu
               e_paise));
    puts("PASS: expected totals and stocks; every rejected sale left all inventory bytes
        unchanged.");return 0;
}
```

Task06/tests.c

```
#include "demo_data.h"
#include <assert.h>
#include <string.h>
#include <limits.h>
#include <stdio.h>
static void rejected(Inventory *inv,unsigned id,unsigned q,Date day,Time time,SaleResult expected){
    Inventory saved=*inv;assert(record_sale(inv,id,q,day,time)==expected);
    assert(!memcmp(&saved,inv,sizeof saved));
}
int main(void){
    Inventory inv;setup_demo(&inv);Date day=inv.business_day;Time now={9,0};
    rejected(&inv,3,1,day,(Time){7,59},UNAVAILABLE);
    rejected(&inv,1,1,(Date){11,11,2026},now,BAD_REQUEST);
    rejected(&inv,1,1,day,(Time){24,0},BAD_REQUEST);
    rejected(&inv,1,1000001,day,now,BAD_REQUEST);
    inv.sale_count=MAX_SALES;rejected(&inv,1,1,day,now,SALES_FULL);inv.sale_count=0;
    inv.items[0].revenue_paise=LLONG_MAX-
        1;rejected(&inv,1,1,day,now,AMOUNT_OVERFLOW);inv.items[0].revenue_paise=0;
    inv.total_revenue=LLONG_MAX-1;rejected(&inv,1,1,day,now,AMOUNT_OVERFLOW);inv.total_revenue=0;
    Item double_water={.id=6,.name="DoubleWater",.price_paise=3500,.kind=COMBO,.data.combo={2,{{1,1
        },{1,2}}}};
    assert(add_item(&inv,&double_water));
    rejected(&inv,6,4,day,now,INSUFFICIENT_STOCK); /* repeated IDs need12, not merely4or8 */
    assert(record_sale(&inv,6,3,day,now)==SALE_OK&&inv.items[0].stock==1);
    assert(inv.total_revenue==10500&&inv.items[0].revenue_paise==0); /* no double revenue */
    Item bad=double_water;bad.id=7;bad.data.combo.parts[0].item_id=4;assert(!add_item(&inv,&bad));
    bad=double_water;bad.id=7;bad.data.combo.parts[0].item_id=99;assert(!add_item(&inv,&bad));
    bad=double_water;bad.id=7;bad.data.combo.parts[0].quantity=0;assert(!add_item(&inv,&bad));
    bad=double_water;bad.id=7;bad.data.combo.count=4;assert(!add_item(&inv,&bad));
    bad=double_water;bad.id=7;bad.kind=(ItemKind)99;assert(!add_item(&inv,&bad));
    assert(!add_item(&inv,&double_water));
    assert(date_valid((Date){29,2,2024})&&!date_valid((Date){29,2,2025}));
    assert(date_ordinal((Date){1,3,2024})-date_ordinal((Date){28,2,2024})==2);
    assert(date_ordinal((Date){1,1,2027})-date_ordinal((Date){30,12,2026})==2);
    setup_demo(&inv);inv.items[0].data.packaged.expiry=day;
    assert(record_sale(&inv,1,1,day,now)==SALE_OK); /* expiry today inclusive */
    inv.items[0].data.packaged.expiry=(Date){9,11,2026};rejected(&inv,1,1,day,now,UNAVAILABLE);
    setup_demo(&inv);inv.items[2].data.cooked.prepared_date=(Date){9,11,2026};
    rejected(&inv,4,1,day,now,UNAVAILABLE); /* invalid component invalidates entire combo */
    setup_demo(&inv);for(unsigned id=6;id<=MAX_ITEMS;id++){
        Item item=inv.items[0];item.id=id;assert(add_item(&inv,&item));
    }
    Item extra=inv.items[0];extra.id=MAX_ITEMS+1;assert(!add_item(&inv,&extra));
    puts("PASS: duplicate component demand aggregated; successful combo revenue counted once.");
    puts("PASS: rejected sale snapshots unchanged for stock, expiry, time, capacity and
        overflow.");
    puts("PASS: invalid recipes/tags/IDs and item capacity rejected; leap/year date boundaries
        verified.");
    puts("PASS: expiry today allowed; expired ingredient and previous-day cooked item
        rejected.");return 0;
}
```

common/date.c

```
#include "date.h"
static bool leap(int y){return y%4==0&&(y%100!=0||y%400==0);}
bool date_valid(Date d){
    const int days[]={31,28,31,30,31,30,31,31,30,31,30,31};
    if(d.year<1900||d.year>9999||d.month<1||d.month>12)return false;
    return d.day>=1&&d.day<=days[d.month-1]+(d.month==2&&leap(d.year));
}
int date_ordinal(Date d){
    const int before[]={0,31,59,90,120,151,181,212,243,273,304,334};
    int y=d.year-1;
    return 365*y+y/4-y/100+y/400+before[d.month-1]+(d.month>2&&leap(d.year))+d.day;
}
```

common/date.h

```
#ifndef CSA02_DATE_H
#define CSA02_DATE_H
#include <stdbool.h>
typedef struct { int day,month,year; } Date;
bool date_valid(Date d);
int date_ordinal(Date d); /* valid dates only; Gregorian day number */
#endif
```